WEEK 4 — AUTOMATION

Wire your stack so it runs without you

اربط أدواتك بحيث تشتغل من دونك

YOUR AI TOOL

Claude

Tuned for Claude — paste each block as a Project instruction or a /command, and let extended thinking expand the checklist before it acts.

مهيّأة لـ Claude — الصق كل جزء كتعليمات Project أو أمر /، وخلّ التفكير الموسّع يوسّع القائمة قبل التنفيذ.

YOUR SYSTEM · ZSH

macOS

Commands use zsh. Install tooling via Homebrew where noted.

الأوامر بـ zsh. ثبّت الأدوات عبر Homebrew حيث يُشار لذلك.

Autopilot Stack: Build Automations That Run Without You

اربط أدواتك بحيث تشتغل من دونك. الفكرة بسيطة: كل مهمة تكررها بيدك كل أسبوع هي مرشّح ليتولاها نظام آلي موثوق. هذا الدليل يعلّمك كيف تبني هذا النظام خطوة بخطوة، مع التركيز على الموثوقية بدل السرعة فقط.

Automation isn't about clever tools. It's about turning repeatable decisions into reliable plumbing, then trusting that plumbing enough to stop watching it. A good automation is boring: it runs, it logs, and it tells you only when something is genuinely wrong. This playbook walks the full lifecycle, from spotting the work to proving the time you saved.

Spot the Work Worth Automating

Not everything deserves a workflow. Automate the wrong thing and you spend more time maintaining it than doing it by hand. Run each candidate through a quick filter:

1. **Frequency.** You do it at least weekly, ideally daily.
2. **Stability.** The steps rarely change. If the process is still in flux, automating it just freezes a bad version.
3. **Rule-clarity.** You can describe the decision in plain sentences. If you can't, the task needs a human judgment step (more on that below).
4. **Cost of error.** Low-to-medium. Start where a mistake is annoying, not catastrophic. Earn trust before you touch money or production data.

The *why*: automation pays back as `time_saved × frequency − maintenance_cost`. Tasks high on frequency and stability but low on error-cost give compounding returns with minimal risk. Write down your top three candidates before building anything.

Map a Trigger → Action Flow

Every automation is the same shape: something *happens* (trigger), the system *does* a sequence of actions, and it *ends* in a known state. Before writing a single line, sketch it on paper.

- **Trigger** — what starts the run: a schedule, an incoming message, a new row, a file landing in a folder, a webhook.
- **Inputs** — what data the run needs, and where it comes from.
- **Steps** — the ordered actions, each small and named.
- **Output** — the artifact or state change that means "done."

Keep each step single-purpose. A step that "fetches, transforms, and emails" is really three steps; when it breaks you won't know which part failed.

```
FLOW: weekly-report
  TRIGGER: schedule(every Monday 08:00)
  STEP 1: fetch raw data from source
  STEP 2: validate shape (expected columns present?)
  STEP 3: transform into summary
  STEP 4: render document
  STEP 5: deliver to recipient
```

```
  ON_SUCCESS: log run_id + duration
  ON_FAILURE: alert owner with step name + error
```

The *why*: naming the failure boundary at design time separates a toy from a tool. When step 2 fails, your alert says "validation failed," not "the report is broken."

Add an AI Step Where Judgment Lives

Pure rules handle structured work. The moment a step needs interpretation, summarization, classification, or drafting, an AI step earns its place. Insert it as one node in the flow, not the whole flow.

Treat it like any other step: typed inputs, a typed output, and a validation gate after it. Never let free-form model output flow straight into an irreversible action. Constrain it.

```
PROMPT TEMPLATE: classify-incoming-message

You are a triage classifier. Read the message below and return STRICT JSON only.

Allowed categories: ["billing", "support", "sales", "spam", "other"]

Rules:
- Choose exactly one category from the allowed list.
- "urgency" is an integer 1 (low) to 5 (critical).
- If the message is empty or unreadable, category = "other", urgency = 1.
- Output nothing except the JSON object.

Message:
"""
{message_body}
"""

Return:
{ "category": "<one allowed value>", "urgency": <1-5>, "reason": "<short>" }
```

The *why*: a closed set of categories and a strict schema make the AI output *checkable*. The next step rejects anything that doesn't parse or falls outside the allowed values, routing those rare cases to a human instead of guessing.

Make It Reliable: Idempotency, Retries, Alerts

This is what separates automations that survive from ones you quietly disable after they double-charge someone. Reliability is designed in, never bolted on.

- **Idempotency.** Running the same step twice must not cause double effects. Tag each unit of work with a stable key (an order id, a date, a hash) and check "did I already process this?" before acting. Re-runs become safe.
- **Retries with backoff.** Transient failures (a timeout, a rate limit) deserve a retry; bugs do not. Retry a few times with increasing delay, then give up loudly. Don't retry forever — that hides the problem and burns quota.
- **Distinguish error types.** A 500 from a service is worth retrying. A 400 from malformed input is not — fix the input.
- **Alert on failure, stay silent on success.** Notifications that fire on every successful run train you to ignore them. Alert only on failure or anomaly (run took 10× longer, output count dropped to zero).
- **Dead-letter the unprocessable.** When something can't be handled after retries, park it somewhere visible with full context so you can inspect and replay it.

```
PSEUDO-FLOW: safe-step
  key = stable_id(item)
  if already_processed(key): return SKIP
  for attempt in 1..3:
    result = do_work(item)
    if result.ok:
      mark_processed(key)
      return OK
    if result.error is permanent:
      send_to_dead_letter(item, result.error)
      return FAIL
    wait(backoff = base * 2^attempt)
  send_to_dead_letter(item, "max retries exceeded")
  alert(owner, step="safe-step", key=key)
```

The *why*: most "the automation went crazy overnight" stories are a missing idempotency check plus an infinite retry. These two patterns prevent the majority of runaway incidents.

Keep Secrets and Credentials Safe

An automation runs unattended, so its credentials are unattended too. Treat them with more care than your own password, because no human is watching when they leak.

1. **Never hardcode.** No keys, tokens, or passwords in the flow definition, code, or logs.
2. **Inject at runtime** from environment variables or a secret store. The automation reads them when it runs and never writes them down.
3. **Scope to the minimum.** Give each automation its own credential with only the permissions that one job needs. A reporting bot should not hold write access to billing.
4. **Validate at startup.** Confirm every required secret is present and fail fast if one is missing — better than discovering it three steps in.
5. **Rotate and revoke.** Plan for rotation, and revoke instantly if a key may be exposed. Scoped per-job credentials make this painless.
6. **Scrub logs.** Mask anything secret-shaped before it touches a log line.

The *why*: an unattended process is the ideal target. Least-privilege, per-job credentials turn a potential breach from "everything is compromised" into "one narrow capability needs rotating."

Measure the Time You Saved

If you can't prove the payback, you can't justify the maintenance — or decide what to automate next. Instrument from day one.

- Log a `run_id`, start time, end time, and outcome for every run.
- Record the manual baseline once: how long the task took by hand, honestly measured.
- Track `runs × baseline_minutes` as gross time saved, then subtract the hours spent maintaining the flow.
- Watch the failure rate over time. A creeping rate signals the underlying process changed and the automation needs attention.

The *why*: automations decay silently as the world around them shifts. A simple dashboard of runs, failures, and minutes saved turns that invisible decay into something you can act on — and gives you the number that justifies building the next one.

Checklist

- [ ] Task is frequent, stable, rule-clear, and low-to-medium error cost
- [ ] Trigger, inputs, ordered steps, and a defined "done" state are written down
- [ ] Each step is single-purpose and individually named
- [ ] AI steps return a strict, validated schema with a closed set of options
- [ ] Every unit of work has an idempotency key and a "already done?" check
- [ ] Retries use backoff, separate transient from permanent errors, and give up loudly
- [ ] Alerts fire only on failure or anomaly; unprocessable items go to a dead-letter queue
- [ ] Secrets are injected at runtime, scoped to least privilege, validated at startup, never logged
- [ ] Every run logs id, duration, and outcome
- [ ] A baseline exists so time saved (minus maintenance) can be measured